In [61]:
```
import torch
print(torch.__version__)

print(sys.version)
print(torch.__version__)
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))

import pandas as pd
import numpy as np
import emoji
import requests
import random
import json

# SKLearn import
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report # For accuracy score
from sklearn.preprocessing import MultiLabelBinarizer # Convert multiple emotion in
```

```
2.11.0+cu128
3.12.14 | packaged by Anaconda, Inc. | (main, Aug 27 2026, 14:37:13) [MSC v.1942 64
bit (AMD64)]
2.11.0+cu128
True
NVIDIA GeForce RTX 4050 Laptop GPU
```

In [62]:
```
df = pd.read_csv('emotion_dataset_labeled_test.csv')
```

In [63]:
```
df.columns
```

Out[63]:
```
Index(['message', 'Happy', 'Sad', 'Angry', 'Frustrated', 'Bored', 'Excited',
       'Jealous', 'Insecure', 'Relieved', 'Neutral'],
      dtype='str')
```

In [64]:
```
df2 = df.copy()
```

df2 = df2.drop(columns=['happy', 'sad', 'angry', 'frustrated', 'bored', 'excited', 'jealous', 'insecure', 'relieved', 'neutral'])

In [65]:
```
df2
```

Out[65]:

	message	Happy	Sad	Angry	Frustrated	Bored	Excited	Jealous	Insecure
0	Hi, Lily. Do you know who am I?	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	You know that? I wanted to test something.\naA...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	Hi Lily. How do you do?	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	Only sad things ever happened today... I watch...	0.0	95.0	0.0	0.0	0.0	0.0	40.0	20.0
4	Ah well, I've never told her my feelings anywa...	0.0	40.0	0.0	0.0	0.0	0.0	0.0	20.0
...	...	...	...	...	...	...	...	...	...
1732	Sometimes I just want to say that this is unfair.	0.0	21.0	81.0	41.0	0.0	0.0	0.0	0.0
1733	I became a parent and suddenly everyone expect...	0.0	45.0	15.0	65.0	0.0	0.0	0.0	30.0
1734	That is probably what I am missing.	0.0	20.0	0.0	40.0	0.0	0.0	0.0	0.0
1735	Maybe I should actually tell my sister how ove...	0.0	45.0	15.0	60.0	0.0	0.0	0.0	10.0
1736	I think I have been trying too hard to look li...	0.0	21.0	0.0	61.0	0.0	0.0	0.0	41.0

1737 rows × 11 columns



In [66]:
```
df2.shape
```

Out[66]:
```
(1737, 11)
```

In [67]:
```
df2.isna().sum()
```

Out[67]:
```
message       0
Happy         0
Sad           0
Angry         0
Frustrated    0
Bored         0
Excited       0
Jealous       0
Insecure      0
Relieved      0
Neutral       0
dtype: int64
```

In [69]:
```
df2_cleaned = df2.dropna()
```

In [70]:
```
df2_cleaned
```

Out[70]:

	message	Happy	Sad	Angry	Frustrated	Bored	Excited	Jealous	Insecure
0	Hi, Lily. Do you know who am I?	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	You know that? I wanted to test something.\naA...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	Hi Lily. How do you do?	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	Only sad things ever happened today... I watch...	0.0	95.0	0.0	0.0	0.0	0.0	40.0	20.0
4	Ah well, I've never told her my feelings anywa...	0.0	40.0	0.0	0.0	0.0	0.0	0.0	20.0
...	...	...	...	...	...	...	...	...	...
1732	Sometimes I just want to say that this is unfair.	0.0	21.0	81.0	41.0	0.0	0.0	0.0	0.0
1733	I became a parent and suddenly everyone expect...	0.0	45.0	15.0	65.0	0.0	0.0	0.0	30.0
1734	That is probably what I am missing.	0.0	20.0	0.0	40.0	0.0	0.0	0.0	0.0
1735	Maybe I should actually tell my sister how ove...	0.0	45.0	15.0	60.0	0.0	0.0	0.0	10.0
1736	I think I have been trying too hard to look li...	0.0	21.0	0.0	61.0	0.0	0.0	0.0	41.0

1737 rows × 11 columns



In [71]:
```
df2_cleaned.isna().sum()
```

Out[71]:
```
message       0
Happy         0
Sad           0
Angry         0
Frustrated    0
Bored         0
Excited       0
Jealous       0
Insecure      0
Relieved      0
Neutral       0
dtype: int64
```

In [72]:
```
df2.columns
```

Out[72]:
```
Index(['message', 'Happy', 'Sad', 'Angry', 'Frustrated', 'Bored', 'Excited',
       'Jealous', 'Insecure', 'Relieved', 'Neutral'],
      dtype='str')
```

In [73]:
```
emotion = ['Happy', 'Sad', 'Angry', 'Frustrated', 'Bored', 'Excited','Jealous', 'In
```

In [74]:
```
df2_cleaned[(df2_cleaned[emotion] < 0) | (df2_cleaned[emotion] > 100)].sum()
```

Out[74]:
```
message
Happy         0.0
Sad           0.0
Angry         0.0
Frustrated    0.0
Bored         0.0
Excited       0.0
Jealous       0.0
Insecure      0.0
Relieved      0.0
Neutral       0.0
dtype: object
```

In [75]:
```
print('Info')
print(df2_cleaned.info())
print('\nDescribe')
print(df2_cleaned.describe())
print('\nShape')
print(df2_cleaned.shape)
```

```
Info
<class 'pandas.DataFrame'>
RangeIndex: 1737 entries, 0 to 1736
Data columns (total 11 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   message     1737 non-null   str
 1   Happy       1737 non-null   float64
 2   Sad         1737 non-null   float64
 3   Angry       1737 non-null   float64
 4   Frustrated  1737 non-null   float64
 5   Bored       1737 non-null   float64
 6   Excited     1737 non-null   float64
 7   Jealous     1737 non-null   float64
 8   Insecure    1737 non-null   float64
 9   Relieved    1737 non-null   float64
 10  Neutral     1737 non-null   float64
dtypes: float64(10), str(1)
memory usage: 149.4 KB
None

Describe
              Happy          Sad        Angry   Frustrated        Bored  \
count  1737.000000  1737.000000  1737.000000  1737.000000  1737.000000
mean      4.128382    14.976396     7.123777    13.576857     0.722510
std      16.387206    24.153551    20.083271    22.155192     5.850393
min       0.000000     0.000000     0.000000     0.000000     0.000000
25%       0.000000     0.000000     0.000000     0.000000     0.000000
50%       0.000000     0.000000     0.000000     0.000000     0.000000
75%       0.000000    21.000000     0.000000    25.000000     0.000000
max     100.000000    95.000000    95.000000    90.000000    95.000000

           Excited      Jealous     Insecure     Relieved      Neutral
count  1737.000000  1737.000000  1737.000000  1737.000000  1737.000000
mean      1.617732     2.002879     7.366149     0.565918    50.367300
std       9.266726    10.948481    18.890467     5.922731    46.382623
min       0.000000     0.000000     0.000000     0.000000     0.000000
25%       0.000000     0.000000     0.000000     0.000000     0.000000
50%       0.000000     0.000000     0.000000     0.000000    40.000000
75%       0.000000     0.000000     0.000000     0.000000   100.000000
max      95.000000    95.000000    95.000000    90.000000   100.000000

Shape
(1737, 11)
```

Checking Phase

In [76]:

```python
count = 0
sampled_data = df2_cleaned.sample(15)

for index, row in sampled_data.iterrows():
    print(f'{index}, {count}')
    print(row['message'])
    print(row[emotion])
```

```
    print()
    count+= 1
```

```
79, 0
ahhh okay thats fine
Happy           0.0
Sad             0.0
Angry           0.0
Frustrated      0.0
Bored           0.0
Excited         0.0
Jealous         0.0
Insecure        0.0
Relieved        0.0
Neutral       100.0
Name: 79, dtype: object

1083, 1
Sometimes I wish I could just not wake up tomorrow.
Happy          0.0
Sad           75.0
Angry          0.0
Frustrated    45.0
Bored          0.0
Excited        0.0
Jealous        0.0
Insecure       0.0
Relieved       0.0
Neutral       25.0
Name: 1083, dtype: object

1323, 2
I just wish I could have gotten the grade they wanted.
Happy          0.0
Sad           70.0
Angry         20.0
Frustrated    80.0
Bored          0.0
Excited        0.0
Jealous        0.0
Insecure      30.0
Relieved       0.0
Neutral        0.0
Name: 1323, dtype: object

1223, 3
What if every relationship ends like this?
Happy          0.0
Sad           65.0
Angry          0.0
Frustrated     0.0
Bored          0.0
Excited        0.0
Jealous        0.0
Insecure       0.0
Relieved       0.0
Neutral       35.0
Name: 1223, dtype: object
```

```
1118, 4
I do not know if anything can be repaired.
Happy          0.0
Sad           41.0
Angry          0.0
Frustrated    61.0
Bored          0.0
Excited        0.0
Jealous        0.0
Insecure       0.0
Relieved       0.0
Neutral        0.0
Name: 1118, dtype: object

411, 5
I know
Happy           0.0
Sad             0.0
Angry           0.0
Frustrated      0.0
Bored           0.0
Excited         0.0
Jealous         0.0
Insecure        0.0
Relieved        0.0
Neutral       100.0
Name: 411, dtype: object

1347, 6
That's exactly it.
Happy           0.0
Sad             0.0
Angry           0.0
Frustrated      0.0
Bored           0.0
Excited         0.0
Jealous         0.0
Insecure        0.0
Relieved        0.0
Neutral       100.0
Name: 1347, dtype: object

1728, 7
I love my family. I just need them to leave me alone for five minutes sometimes.
Happy         45.0
Sad            0.0
Angry         25.0
Frustrated    35.0
Bored          0.0
Excited        0.0
Jealous        0.0
Insecure       0.0
Relieved       0.0
Neutral       15.0
Name: 1728, dtype: object
```

```
584, 8
Obviously
Happy           0.0
Sad             0.0
Angry           0.0
Frustrated      0.0
Bored           0.0
Excited         0.0
Jealous         0.0
Insecure        0.0
Relieved        0.0
Neutral       100.0
Name: 584, dtype: object

733, 9
Maybe fried rice
Happy           0.0
Sad             0.0
Angry           0.0
Frustrated      0.0
Bored           0.0
Excited         0.0
Jealous         0.0
Insecure        0.0
Relieved        0.0
Neutral       100.0
Name: 733, dtype: object

56, 10
My friend surprised me with my favorite food after class  I didnt even tell him I wa
nted it
Happy         75.0
Sad            0.0
Angry          0.0
Frustrated     0.0
Bored          0.0
Excited       30.0
Jealous        0.0
Insecure       0.0
Relieved       0.0
Neutral        0.0
Name: 56, dtype: object

937, 11
I wish there were a way to know for certain.
Happy          0.0
Sad           20.0
Angry          0.0
Frustrated    40.0
Bored          0.0
Excited        0.0
Jealous        0.0
Insecure      20.0
Relieved       0.0
Neutral        0.0
Name: 937, dtype: object
```

```
91, 12
Maybe ill call my friend
Happy           0.0
Sad             0.0
Angry           0.0
Frustrated      0.0
Bored           0.0
Excited         0.0
Jealous         0.0
Insecure        0.0
Relieved        0.0
Neutral       100.0
Name: 91, dtype: object

207, 13
I want to buy something
Happy           0.0
Sad             0.0
Angry           0.0
Frustrated      0.0
Bored           0.0
Excited         0.0
Jealous         0.0
Insecure        0.0
Relieved        0.0
Neutral       100.0
Name: 207, dtype: object

1252, 14
She's prettier than me anyway.
Happy          0.0
Sad            0.0
Angry          0.0
Frustrated     0.0
Bored          0.0
Excited        0.0
Jealous       45.0
Insecure      35.0
Relieved       0.0
Neutral        0.0
Name: 1252, dtype: object
```

In [77]:
```
emotion
```

Out[77]:
```
['Happy',
 'Sad',
 'Angry',
 'Frustrated',
 'Bored',
 'Excited',
 'Jealous',
 'Insecure',
 'Relieved',
 'Neutral']
```

In [78]:
```
df2_cleaned[(df2_cleaned[emotion] == 0).all(axis=1)]
```

Out[78]:

	message	Happy	Sad	Angry	Frustrated	Bored	Excited	Jealous	Insecure
1	You know that? I wanted to test something.\naA...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
7	Hihi. Debugging time~	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
88	im actually panicking right now	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
92	Lily what if i just steal the thing i want lol	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
134	Sorry XD	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
151	I dont want to make things awkward	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
156	Thats probably what I need to hear	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
231	What if I dont	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
396	I think I was just panicking	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
404	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
416	That feels less scary	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
453	Aww~ But I really need the flirting tips now. ...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
491	There is a game I really want	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
521	Someone I like was talking to someone else	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
534	Honestly no	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
630	Very much	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
652	Lol	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
689	It really did	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
710	I am nervous about tomorrow	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

	message	Happy	Sad	Angry	Frustrated	Bored	Excited	Jealous	Insecure
918	That is strangely comforting. I had imagined a...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1013	I know. I just keep imagining what would happen.	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1070	That distinction actually makes me feel less w...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1299	Sometimes I imagine saying something really cr...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1302	It feels less intense when I talk about it.	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1476	Someone complimented me today and I immediatel...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1701	Exactly. They call it helping.	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Identify problematic data and drop it

In this case, a data where all the emotion is zero, while it shouldn't be so.

df2_cleaned.loc[1196, ["message"] + emotion]print(df2_cleaned.loc[1196, emotion] == 0)

In [79]:
```
# df2_cleaned = df2_cleaned.drop(index=1196)
```

Entering machine learning

In [80]:
```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

# Regressor model / Multiple model for testing purpose
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
```

In [81]:
```
X = df2_cleaned.drop(columns=emotion)
y = df2_cleaned[emotion]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    train_size=0.8, # Since data size is small, more training is better
    random_state=12
)
```

In [82]:
```
tfidf_vector = TfidfVectorizer(
    stop_words='english',
    max_features=5000
)

X_train_tfidf = tfidf_vector.fit_transform(
    X_train['message']
)

print(X_train_tfidf.shape)
```

```
(1389, 1560)
```

In [83]:
```
model_lr = LinearRegression()
model_rf = RandomForestRegressor()
```

In [84]:
```
model_lr.fit(X_train_tfidf, y_train)
model_rf.fit(X_train_tfidf, y_train)
```

Out[84]:

▾ RandomForestRegressor ⓘ ?

▸ Parameters

▸ Fitted attributes

In [85]:
```
X_test_tfidf = tfidf_vector.transform(X_test['message'])
print(X_test_tfidf.shape)
```

```
(348, 1560)
```

In [86]:
```
y_pred_lr = model_lr.predict(X_test_tfidf)
y_pred_rf = model_rf.predict(X_test_tfidf)
```

In [87]:
```
# print(y_pred[:5])
```

In [ ]:
```

```

Prediction result evaluation

In [88]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, clas
```

In [89]:
```
# Linear Regressor
mae_lr = mean_absolute_error(y_test, y_pred_lr)
mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)
```

```
rmse_lr = np.sqrt(mse_lr)
print('Linear Regressor Result')
print(f'Mean Absolute Error: {mae_lr}\n')
print(f"Mean Square Error: {mse_lr}\n")
print(f"R2 Score: {r2_lr}\n")
print(f"Root Mean Square Error: {rmse_lr}")
```

```
Linear Regressor Result
Mean Absolute Error: 14.581883712983137

Mean Square Error: 758.6465122247876

R2 Score: -0.8427118302797607

Root Mean Square Error: 27.54353848409437
```

In [90]:
```
# Random Forest Regressor
mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
print('Random Forest Regressor Result')
print(f'Mean Absolute Error: {mae_rf}\n')
print(f"Mean Square Error: {mse_rf}\n")
print(f"R2 Score: {r2_rf}\n")
print(f"Root Mean Square Error: {rmse_rf}")
```

```
Random Forest Regressor Result
Mean Absolute Error: 8.967262523173058

Mean Square Error: 377.7667440286865

R2 Score: 0.12497722381379509

Root Mean Square Error: 19.436222473224742
```

In [60]:
```
print(classification_report(
    y_test_class,
    y_pred_class,
    target_names=emotion,
    zero_division=0
))
```

	precision	recall	f1-score	support
Happy	0.00	0.00	0.00	5
Sad	0.54	0.23	0.33	30
Angry	1.00	0.33	0.50	9
Frustrated	0.53	0.31	0.39	32
Bored	0.00	0.00	0.00	3
Excited	0.00	0.00	0.00	3
Jealous	0.00	0.00	0.00	3
Insecure	0.00	0.00	0.00	9
Relieved	0.00	0.00	0.00	2
Neutral	0.61	0.96	0.75	55
micro avg	0.59	0.48	0.53	151
macro avg	0.27	0.18	0.20	151
weighted avg	0.50	0.48	0.45	151
samples avg	0.61	0.60	0.58	151

In [ ]:

Attempting to use model from Hugging Face

In [37]:
```
# Transformer
import transformers
from transformers import AutoTokenizer
from transformers import XLMRobertaModel # AI Model download

# Import/install training model (XLM-RoBERTa)
model_name = "FacebookAI/xlm-roberta-base"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = XLMRobertaModel.from_pretrained(model_name)
```

```
C:\Users\ASUS\.conda\envs\lily-ai\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IP
rogress not found. Please update jupyter and ipywidgets. See https://ipywidgets.read
thedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOK
EN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 870.01it/s]
[transformers] XLMRobertaModel LOAD REPORT from: FacebookAI/xlm-roberta-base
Key                        | Status     |  |
---------------------------+------------+--+-
lm_head.layer_norm.bias    | UNEXPECTED |  |
lm_head.dense.weight       | UNEXPECTED |  |
lm_head.dense.bias         | UNEXPECTED |  |
lm_head.layer_norm.weight  | UNEXPECTED |  |
lm_head.bias               | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok
if you expect identical arch.
```

In [38]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    train_size=0.8,
    random_state=12
)
```

In [39]:
```
print(f"Training Message: {len(X_train)}")
print(f"Testing Message: {len(X_test)}")
print(f"Training Target: {(y_train.shape)}")
print(f"Testing Target: {(y_test.shape)}")
```

```
Training Message: 400
Testing Message: 100
Training Target: (400, 10)
Testing Target: (100, 10)
```

In [40]:
```
train_encodings = tokenizer(
    X_train['message'].tolist(),
    truncation=True,
    padding=True,
    max_length=128,
    return_tensors="pt"
)
```

In [41]:
```
print(train_encodings.keys())
print(train_encodings["input_ids"].shape)
print(train_encodings["attention_mask"].shape)
```

```
KeysView({'input_ids': tensor([[    0,  1061,    87,  ...,     1,     1,     1],
        [    0,    87,   100,  ...,     1,     1,     1],
        [    0,    87,    25,  ...,     1,     1,     1],
        ...,
        [    0,    87,    25,  ...,     1,     1,     1],
        [    0,  2646, 13909,  ...,     1,     1,     1],
        [    0, 83425,    87,  ...,     1,     1,     1]]), 'attention_mask': tensor
([[1, 1, 1,  ..., 0, 0, 0],
        [1, 1, 1,  ..., 0, 0, 0],
        [1, 1, 1,  ..., 0, 0, 0],
        ...,
        [1, 1, 1,  ..., 0, 0, 0],
        [1, 1, 1,  ..., 0, 0, 0],
        [1, 1, 1,  ..., 0, 0, 0]])})
torch.Size([400, 74])
torch.Size([400, 74])
```

In [42]:
```
y_train_tensor = torch.tensor(
    y_train.values,
    dtype=torch.float32
)

y_test_tensor = torch.tensor(
    y_test.values,
    dtype=torch.float32
)
```

In [43]:
```
print(y_train_tensor.shape)
print(y_test_tensor.shape)
```

```
torch.Size([400, 10])
torch.Size([100, 10])
```

In [44]:
```
from transformers import XLMRobertaForSequenceClassification
```

In [45]:
```
model = XLMRobertaForSequenceClassification.from_pretrained(
    model_name,
    num_labels=10,
    problem_type="regression"
)
```

```
Loading weights: 100%|██████████| 197/197 [00:00<00:00, 622.40it/s]
[transformers] XLMRobertaForSequenceClassification LOAD REPORT from: FacebookAI/xlm-roberta-base
Key                          | Status     |
-----------------------------+------------+-
lm_head.bias                 | UNEXPECTED |
lm_head.layer_norm.bias      | UNEXPECTED |
lm_head.dense.weight         | UNEXPECTED |
roberta.pooler.dense.bias    | UNEXPECTED |
lm_head.dense.bias           | UNEXPECTED |
lm_head.layer_norm.weight    | UNEXPECTED |
roberta.pooler.dense.weight  | UNEXPECTED |
classifier.out_proj.weight   | MISSING    |
classifier.out_proj.bias     | MISSING    |
classifier.dense.weight      | MISSING    |
classifier.dense.bias        | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING:      those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

In [46]:
```
print(type(model))
print(model.config.num_labels)
print(model.config.problem_type)
```

```
<class 'transformers.models.xlm_roberta.modeling_xlm_roberta.XLMRobertaForSequenceClassification'>
10
regression
```

In [49]:
```
test_encodings = tokenizer(
    X_test['message'].tolist(),
    truncation=True,
    padding=True,
    max_length=128,
    return_tensors='pt'
)
```

In [52]:
```
from torch.utils.data import Dataset
```

```
class EmotionDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        item = {
            key: value[idx]
            for key, value in self.encodings.items()
        }
        item["labels"] = self.labels[idx]
        return item
```

In [53]:
```
train_dataset = EmotionDataset(
    train_encodings,
    y_train_tensor
)

test_dataset = EmotionDataset(
    test_encodings,
    y_test_tensor
)
```

In [54]:
```
print(len(train_dataset))
print(train_dataset[0].keys())
print(train_dataset[0]["labels"].shape)
```

```
400
dict_keys(['input_ids', 'attention_mask', 'labels'])
torch.Size([10])
```

In [ ]: